

Cos'è SQL

SQL (*Structured Query Language*) è il **linguaggio standard per la gestione e l'interrogazione dei database relazionali**.

Serve per **creare, modificare, interrogare e gestire** i dati all'interno di un **RDBMS** (*Relational Database Management System*), come **Microsoft SQL Server, MySQL, PostgreSQL**, ecc.

Database relazionale

Un **database relazionale** organizza i dati in **tabelle** costituite da **righe (record)** e **colonne (campi)**.

- Ogni **riga** rappresenta un record (una singola istanza di dato).
- Ogni **colonna** rappresenta un attributo (una proprietà del record).

Il termine “*relazionale*” deriva dal concetto matematico di **relazione tra insiemi**:

una tabella può essere vista come un **insieme di tuple** (righe) con la stessa struttura (le colonne).

Le relazioni tra le tabelle si stabiliscono tramite **chiavi primarie (PRIMARY KEY)** e **chiavi esterne (FOREIGN KEY)**, che consentono di collegare logicamente i dati.

T-SQL (Transact-SQL)

T-SQL (*Transact-SQL*) è l'**estensione proprietaria di Microsoft** del linguaggio SQL standard, utilizzata in **Microsoft SQL Server** e **Azure SQL Database**.

È un **linguaggio dichiarativo**, cioè **si specifica cosa si vuole ottenere** e non come deve essere eseguito.

Esempio:

Quando si scrive una query SELECT, **non si dice al database come cercare i dati**, ma solo **quali dati** vogliamo vedere.

Transazioni

Una **transazione** è un **blocco logico di operazioni** che vengono eseguite come un'unica unità.

Serve per garantire l'**integrità dei dati**: o tutte le operazioni vengono completate correttamente (**COMMIT**), oppure nessuna di esse viene applicata (**ROLLBACK**).

Standardizzazione

Il linguaggio SQL è definito da due organismi di standardizzazione:

- **ISO** (International Organization for Standardization)
- **ANSI** (American National Standards Institute)

Questo significa che la **sintassi di base** è comune a tutti i database relazionali, ma ogni produttore (come Microsoft, Oracle, PostgreSQL, ecc.) può aggiungere **estensioni proprietarie**, come appunto **T-SQL**.

Il linguaggio SQL si divide in varie **sotto-lingue**, ognuna con scopi specifici.

1. DDL — Data Definition Language

Serve per **definire la struttura** del database (tabelle, vincoli, viste, indici, ecc.).

Comandi principali:

- CREATE → crea oggetti del database
- ALTER → modifica la struttura di un oggetto
- DROP → elimina un oggetto
- TRUNCATE → svuota una tabella senza eliminare la struttura

2. DML — Data Manipulation Language

Serve per **manipolare i dati** già presenti nelle tabelle.

Comandi principali:

- SELECT → interroga i dati
- INSERT → inserisce nuovi record
- UPDATE → modifica record esistenti
- DELETE → elimina record

Operatori nelle query

Quando si scrivono **condizioni** nelle query (ad esempio nel WHERE), si utilizzano **operatori**.

Gli operatori più comuni sono:

- **Operatori di confronto:** =, <>, >, <, >=, <=
- **Operatori logici:** AND, OR, NOT
- **Operatori speciali:** BETWEEN, IN, LIKE, IS NULL

Differenza tra DELETE e TRUNCATE

Entrambi i comandi servono per **rimuovere dati da una tabella**, ma agiscono in modo molto diverso:

Comando	Descrizione	Effetto su struttura	Transazione	Performance
DELETE	Elimina una o più righe una alla volta , valutando una condizione nel WHERE.	Non modifica la struttura della tabella.	È loggato : ogni riga cancellata viene registrata nel log transazionale (più lento ma sicuro).	Più lento per grandi quantità di dati.
TRUNCATE	Cancella tutti i dati della tabella in modo diretto e veloce.	Reimposta anche la struttura interna (es. contatore IDENTITY).	È parzialmente loggato : registra solo la deallocazione delle pagine dati.	Molto più veloce.

Nota importante

- TRUNCATE **non può avere una clausola WHERE**, mentre DELETE sì.
- Se una tabella è **referenziata da una foreign key**, TRUNCATE **non è consentito**.
- Dopo un TRUNCATE, la tabella risulta **vuota**, ma ancora **esistente**.

Uso dell'operatore * nel SELECT

L'operatore * serve a **selezionare tutte le colonne** di una tabella, ad esempio:

```
SELECT * FROM Clienti;
```

Tuttavia, è **sconsigliato (deprecato in contesti professionali)**, perché:

- Se in futuro aggiungi o rimuovi colonne, la query restituirà un **numero diverso di campi**, causando errori nelle applicazioni che si aspettano una struttura precisa.
- Migliora la leggibilità e le performance specificare sempre le colonne necessarie.

Buona pratica:

```
SELECT Nome, Cognome, Età FROM Clienti;
```

Alias e spazi nei nomi delle colonne

Quando si assegna un **alias** a una colonna, non si possono usare spazi **senza racchiudere il nome tra parentesi quadre o doppi apici**.

Esempio errato

```
SELECT Nome AS Cognome Hobbit FROM Hobbits;
```

Esempio corretto

```
SELECT Nome AS NomeHobbit,  
       Cognome AS [Cognome Hobbit]  
FROM Hobbits;
```

Le **parentesi quadre** [] fungono da “**zona franca**” per nomi contenenti spazi o parole riservate del linguaggio.

Clausola IN

Quando una condizione deve essere confrontata con **più valori della stessa colonna**, invece di scrivere tante condizioni concatenate con OR, si usa l’operatore **IN**.

Esempio:

```
SELECT *  
FROM Clienti  
WHERE Città IN ('Roma', 'Milano', 'Napoli');
```

Equivalente a:

```
WHERE Città = 'Roma' OR Città = 'Milano' OR Città = 'Napoli';
```

Più leggibile, più veloce, più mantenibile.

Ordinamento dei risultati (ORDER BY)

Serve per **ordinare** i dati restituiti da una query.

```
SELECT * FROM Clienti  
ORDER BY Cognome ASC;
```

- **ASC** → ordine crescente (default)
- **DESC** → ordine decrescente
- **NULL** → non influisce sull’ordinamento, viene posizionato per ultimo (ma è configurabile)

Esempio:

```
SELECT Nome, Età  
FROM Persone  
ORDER BY Età DESC;
```

Uso degli alias con WHERE e ORDER BY

- Gli **alias non possono essere usati nel WHERE**, perché la clausola WHERE viene eseguita **prima** che l'alias venga assegnato.
- Gli **alias possono essere usati nel ORDER BY**, poiché questa clausola agisce **dopo** la selezione dei dati.

Esempio:

```
SELECT Nome + ' ' + Cognome AS [Nome Completo]
FROM Clienti
ORDER BY [Nome Completo]; -- valido
```

Concatenazione di colonne

Per unire più campi in un'unica colonna di output si possono usare:

1. L'operatore +

```
SELECT Nome + ' ' + Cognome AS [Nome Completo]
FROM Clienti;
```

2. La funzione **CONCAT()**

```
SELECT CONCAT(Nome, ' ', Cognome) AS [Nome Completo]
FROM Clienti;
```

Differenza

- **CONCAT()** effettua un **cast implicito a stringa** dei valori (quindi funziona anche con numeri o NULL).
- L'operatore + invece può restituire NULL se uno dei valori concatenati è NULL.

Buona pratica: **usare sempre CONCAT()** per evitare errori di tipo o di valore nullo.

Alias obbligatori per espressioni

Ogni volta che si esegue un'operazione o funzione su una colonna (non un semplice riferimento diretto), è **obbligatorio assegnare un alias**, altrimenti nel risultato apparirà il nome di default (**No column name**).

Esempio:

```
SELECT CONCAT(Nome, ' ', Cognome) AS [Nome Completo]
FROM Clienti;
```

Limite dei risultati — TOP

Per **limitare il numero di righe restituite**, si utilizza la clausola **TOP**:

```
SELECT TOP (10) *
FROM Clienti;
```

Restituisce solo le **prime 10 righe** in base all'ordinamento specificato (se presente un ORDER BY).

Schema dbo (Database Owner)

Ogni tabella o oggetto in SQL Server appartiene a uno **schema**.

Se non ne viene specificato uno, viene assegnato automaticamente lo **schema predefinito** chiamato **dbo** (*database owner*).

Esempio:

```
CREATE TABLE dbo.Clienti (...);
```

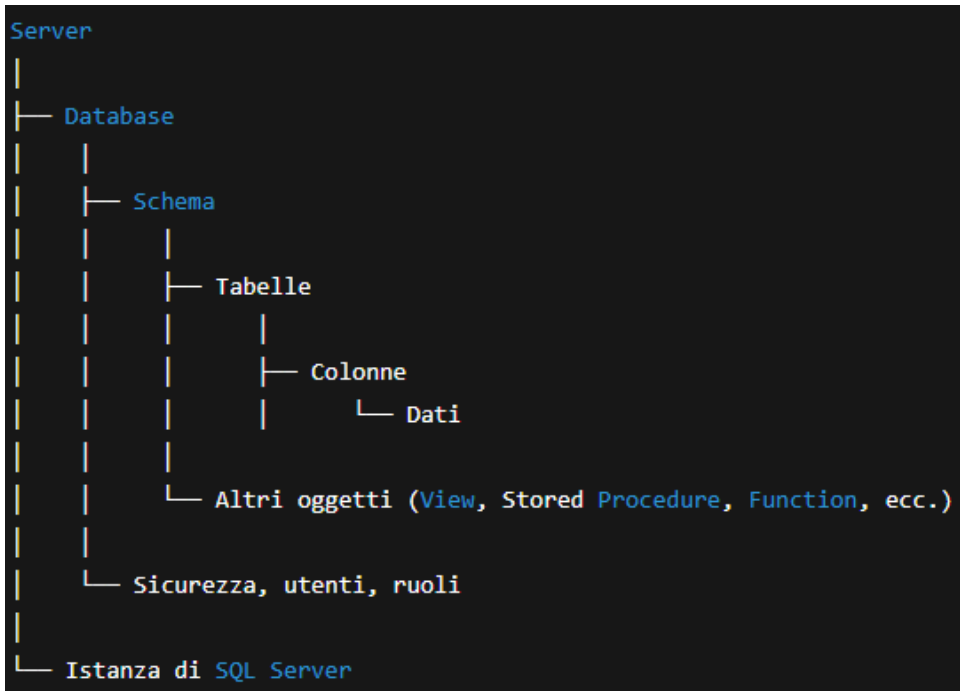
Buone pratiche sugli schemi:

- È **consigliato specificare sempre lo schema**, per chiarezza e sicurezza.
- Gli schemi servono a **raggruppare tabelle correlate** (es. vendite.Clienti, vendite.Ordini).
- Sono utili anche per **gestire i permessi** in modo più granulare.
- Non usare sempre dbo per default: crea schemi **logici** per aree funzionali.

Gerarchia logica del database in SQL Server

In SQL Server, tutti gli elementi che compongono un database seguono una **gerarchia logica ben definita**.

Comprenderla è fondamentale per lavorare correttamente con oggetti, permessi e organizzazione dei dati.



Spiegazione della gerarchia

1. Server

È il livello più alto.

Rappresenta l'**istanza di SQL Server** installata su una macchina fisica o virtuale.

All'interno del server possono coesistere più **database indipendenti**.

2. Database

È un contenitore logico che **organizza i dati e gli oggetti correlati** (tabelle, viste, stored procedure, funzioni, ecc.).

Ogni database ha la propria **struttura, utenti, ruoli e sicurezza**.

Esempi: master, tempdb, msdb, oppure database creati dall'utente come Scuola, Negozio, LotrDB.

3. Schema

È un livello **intermedio di organizzazione** dentro un database.

Serve per **raggruppare oggetti correlati** (tabelle, viste, procedure, funzioni, ecc.) e per **gestire i permessi** in modo più efficiente.

- Ogni tabella appartiene a **uno schema**.

- b. Se non viene specificato, lo schema predefinito è **dbo** (*database owner*).
- c. È buona pratica creare schemi specifici (es. **lotr**, vendite, anagrafe) per **migliorare la chiarezza e la sicurezza**.

4. Tabelle

Le **tabelle** contengono i dati strutturati.

Ogni tabella ha un nome univoco all'interno di uno schema e contiene **colonne** e **righe**.

Esempio:

`lotr.Hobbits` → indica la tabella `Hobbits` appartenente allo schema `lotr`.

5. Colonne

Ogni tabella è composta da colonne che **definiscono la struttura** dei dati (nome, tipo, vincoli, ecc.).

Esempio: `Nome NVARCHAR(50), Età INT, DataNascita DATE`.

6. Dati

Sono i **valori effettivi** memorizzati nelle righe della tabella.

Ogni riga rappresenta un **record**.

Creazione di uno schema

Per creare un nuovo schema nel database, si utilizza il comando **CREATE SCHEMA**:

```
CREATE SCHEMA lotr;  
GO
```

- Lo schema **lotr** ora è disponibile per creare oggetti al suo interno.
- Si può ora creare una tabella con:

```
CREATE TABLE lotr.Hobbits (  
    IdHobbit INT PRIMARY KEY,  
    Nome NVARCHAR(50),  
    Età INT  
);
```

Spostare una tabella in un altro schema

Se una tabella esiste già all'interno di uno schema (es. `dbo`) e vuoi **trasferirla in un nuovo schema**, si usa il comando **ALTER SCHEMA ... TRANSFER**.

Esempio:

```
ALTER SCHEMA lotr TRANSFER dbo.Hobbits;
```


Cosa succede:

- La tabella Hobbits, inizialmente nello schema **dbo**, viene spostata nello schema **lotr**.
- Dopo il trasferimento, il nome completo della tabella diventa:

lotr.Hobbits

- Tutti i dati e la struttura restano invariati, ma cambia l'**appartenenza logica** della tabella.

Importanza degli schemi

Gli **schemi** non sono semplici contenitori:

servono a **organizzare** e **proteggere** gli oggetti del database in modo ordinato e scalabile.

Vantaggi principali:

- Migliore **organizzazione** delle tabelle (es. vendite.Clienti, magazzino.Prodotti).
- **Gestione dei permessi** per gruppo di oggetti (puoi concedere accesso solo a uno schema intero).
- Evita conflitti di nomi: puoi avere due tabelle con lo stesso nome in schemi diversi (lotr.Hobbits e dbo.Hobbits).
- Maggiore **manutenibilità** in database di grandi dimensioni.

Il concetto di NULL

In SQL, **NULL** rappresenta l'**assenza di un valore** (non è uno zero, né una stringa vuota).

È un **valore sconosciuto** o **non definito**.

Importante: qualunque **operazione logica o aritmetica** che coinvolga un valore NULL **restituisce FALSE o UNKNOWN**.

Esempio:

```
SELECT 1 + NULL; -- Restituisce NULL
SELECT NULL = NULL; -- Restituisce UNKNOWN
```

Per gestire i valori nulli, SQL Server mette a disposizione la funzione **ISNULL()**, che permette di **sostituire un valore NULL con un valore di default**.

Sintassi:

ISNULL(espressione, valore_sostitutivo)

Esempio:

```
SELECT ISNULL(Cognome, '-') AS CognomePulito
FROM lotr.Hobbits;
```

In questo modo, se Cognome è NULL, verrà mostrato un trattino ' - '.

Esercizi di Query su NULL e Ordinamento

1-Sostituire i NULL e filtrare i risultati

```
SELECT nome,
       cognome,
       ISNULL(Cognome, '-') AS IsNullCognome
FROM lotr.Hobbits
WHERE ISNULL(Cognome, '-') <> 'Baggins'
ORDER BY nome;
```

- **ISNULL()** sostituisce i NULL con ' - ' nel risultato e nel filtro.
- **WHERE ISNULL(Cognome, '-') <> 'Baggins'** evita di escludere i valori NULL in modo implicito.

2-Trovare cognomi che alfabeticamente precedono “Brandybuck”

```
SELECT DISTINCT cognome AS surname
FROM lotr.Hobbit
WHERE cognome <= 'Brandybuck'
ORDER BY surname ASC;
```

- **DISTINCT** elimina i duplicati.
- **ORDER BY ASC** ordina alfabeticamente in modo crescente.

Le stringhe vengono confrontate **in base all’ordine alfabetico** del set di caratteri definito (collation). In SQL Server, le lettere maiuscole/minuscole possono influire sul confronto **solo se la collation è case-sensitive**.

3-Mostrare i primi 4 hobbit ordinati per nome

```
SELECT TOP 4 nome,
       ISNULL(cognome, '-') AS cognome
FROM lotr.Hobbits
ORDER BY nome;
```

- **TOP 4** restituisce solo le **prime 4 righe**.
- I cognomi nulli vengono sostituiti da un **trattino**.
- **ORDER BY** ordina in base al nome.

4-Concatenare nome e cognome in un'unica colonna

Se vogliamo visualizzare nome e cognome insieme, possiamo concatenarli con l'operatore +:

```
SELECT nome + ' ' + ISNULL(cognome, '-') AS NomeCompleto
FROM lotr.Hobbits
WHERE cognome BETWEEN 'Baggins' AND 'Took+';
```

- + ' ' aggiunge uno spazio tra nome e cognome.
- **BETWEEN** include i limiti estremi del range ('Baggins' e 'Took+').
- **ISNULL()** gestisce i cognomi mancanti.

Regole e buone pratiche su DISTINCT e ORDER BY

Attenzione:

Quando utilizzi **DISTINCT** insieme a **ORDER BY**, i campi presenti in **ORDER BY** devono essere inclusi nella selezione o corrispondere a una colonna del **DISTINCT**.

Esempio corretto:

```
SELECT DISTINCT cognome
FROM lotr.Hobbits
ORDER BY cognome;
```

Esempio errato:

```
SELECT DISTINCT cognome
FROM lotr.Hobbits
ORDER BY nome; -- 'nome' non è presente nella SELECT DISTINCT
```

Uso dell'Alias

L'**alias** (assegnato con AS) serve a **rinominare colonne o tabelle** per rendere più leggibili le query.

- Posso usare l'alias nel **GROUP BY**:

```
SELECT ISNULL(cognome, '-') AS Surname, COUNT(*) AS Numero
FROM lotr.Hobbits
```

```
GROUP BY ISNULL(cognome, ' -');
```

Qui **non è obbligatorio** usare l'alias nel GROUP BY, ma è **consigliato** per chiarezza.

Operatore LIKE

L'operatore **LIKE** serve per **effettuare ricerche testuali parziali** all'interno delle colonne di tipo stringa (CHAR, VARCHAR, NVARCHAR, ecc.).

È particolarmente utile per **filtrare dati che contengono, iniziano o finiscono con determinate lettere o parole**.

Sintassi base

```
SELECT colonna
FROM tabella
WHERE colonna LIKE 'modello_di_ricerca';
```

Caratteri speciali usati con LIKE

Simbolo	Significato	Esempio	Descrizione
%	Qualsiasi sequenza di caratteri (anche vuota)	'a%'	Tutte le stringhe che iniziano con “a”
_	Un singolo carattere qualsiasi	'a_'	Tutte le stringhe di due caratteri che iniziano con “a”
[]	Un set o intervallo di caratteri	'[a-c]%'	Stringhe che iniziano con a, b o c
[^]	Esclude un set di caratteri	'[^a-c]%'	Stringhe che non iniziano con a, b o c

Esempi pratici

```
-- Tutti i nomi che iniziano con 'Sam'
SELECT nome
FROM lotr.Hobbits
WHERE nome LIKE 'Sam%';
```

```
-- Tutti i cognomi che terminano con 'ins'
SELECT cognome
FROM lotr.Hobbits
WHERE cognome LIKE '%ins';
```

```
-- Tutti i nomi di quattro lettere che iniziano con 'F'
SELECT nome
FROM lotr.Hobbits
```

```
WHERE nome LIKE 'F____';
```

Suggerimento:

In SQL Server, il comportamento del LIKE dipende dalla **collation** della colonna:

- Se è **case-insensitive** (predefinito), LIKE 'sam%' e LIKE 'Sam%' restituiscono gli stessi risultati.
- Se è **case-sensitive**, le maiuscole e minuscole fanno la differenza.

Comando CASE WHEN

Il costrutto **CASE WHEN** è l'equivalente di un “if-then-else” nei linguaggi di programmazione. Permette di **valutare condizioni** e **restituire valori differenti** in base al risultato.

Sintassi generale

```
CASE
  WHEN condizione1 THEN valore1
  WHEN condizione2 THEN valore2
  ...
  ELSE valore_default
END
```

Può essere usato **all'interno di una SELECT**, ma anche in **ORDER BY**, **GROUP BY** o **HAVING**.

Esempio pratico

```
SELECT OrderID,
       Quantity,
       CASE
         WHEN Quantity > 30 THEN 'The quantity is greater than 30'
         WHEN Quantity = 30 THEN 'The quantity is 30'
         ELSE 'The quantity is under 30'
       END AS QuantityText
FROM OrderDetails;
```

Spiegazione:

```
CASE valuta la colonna Quantity per ogni riga.+
  WHEN età < 18 THEN 'Minorenne'
  WHEN età BETWEEN 18 AND 65 THEN 'Adulto'
  ELSE 'Anziano'
END AS FasciaEtà
```

```
FROM lotr.Hobbits;
```

Classifica gli hobbit in base all'età.

2-Usare CASE per calcolare valori numerici

```
SELECT nome,  
       stipendio,  
       CASE  
         WHEN stipendio < 1000 THEN stipendio * 1.05  
         WHEN stipendio BETWEEN 1000 AND 2000 THEN stipendio * 1.10  
         ELSE stipendio * 1.15  
       END AS StipendioAggiornato  
FROM dipendenti;
```

Calcola un aumento proporzionale in base alla fascia di stipendio.

3-Usare CASE in un ORDER BY

```
SELECT nome, cognome, ruolo  
FROM dipendenti  
ORDER BY  
  CASE  
    WHEN ruolo = 'Manager' THEN 1  
    WHEN ruolo = 'Analyst' THEN 2  
    ELSE 3  
  END;
```

Ordina i risultati **in base a una logica personalizzata**, non alfabetica.

Tipi di CASE

Tipo	Descrizione	Esempio
Searched CASE	Ogni WHEN valuta una condizione booleana completa	CASE WHEN Quantità > 10 THEN ...
Simple CASE	Confronta un singolo valore con più possibilità	CASE Colore WHEN 'Rosso' THEN ...

Esempio di Simple CASE:

```
SELECT Colore,  
       CASE Colore  
         WHEN 'Rosso' THEN 'Caldo'  
         WHEN 'Blu' THEN 'Freddo'  
         ELSE 'Neutro'  
       END AS TipoColore  
FROM Prodotti;
```

T-SQL – CTE (Common Table Expressions) e funzione IIF

CTE – Common Table Expressions

Una **CTE (Common Table Expression)** è una **query temporanea nominata** che può essere **utilizzata come se fosse una tabella o una vista** all'interno di un'istruzione SELECT, INSERT, UPDATE o DELETE.

In pratica, una CTE ci permette di:

- **Organizzare il codice SQL** rendendolo più leggibile e modulare.
- **Riutilizzare** il risultato di una query **nella stessa istruzione**.
- **Simulare una tabella** temporanea in memoria, accessibile solo per la durata dell'istruzione SQL.

Sintassi base

```
WITH NomeCTE AS (  
    SELECT Colonne  
    FROM Tabella  
    WHERE Condizioni  
)  
SELECT *  
FROM NomeCTE  
WHERE ...
```

Esempio pratico

```
WITH EmployeeCTE AS (  
    SELECT  
        EmployeeID,  
        JobTitle,  
        CASE  
            WHEN VacationHours < 10 THEN 'Needs Vacation'  
            ELSE 'Sufficient Vacation'  
        END AS VacationStatus  
    FROM HumanResources.Employee  
)  
SELECT *  
FROM EmployeeCTE  
WHERE VacationStatus = 'Needs Vacation';
```

Spiegazione dell'esempio:

1. La query **dentro la CTE** (EmployeeCTE) calcola per ogni dipendente se ha **abbastanza ore di ferie** o se ne ha **poche**.
2. Nel **secondo SELECT**, la CTE viene trattata **come una tabella** e si selezionano solo i record con `VacationStatus = 'Needs Vacation'`.

Caratteristiche delle CTE

Aspetto	Descrizione
Durata	Una CTE esiste solo per la durata della query in cui è dichiarata. Dopo l'esecuzione, non viene salvata nel database.
Nome colonne	È obbligatorio assegnare un nome a ogni colonna nella definizione della CTE, soprattutto quando la query interna contiene espressioni o calcoli .

Riutilizzo	Una CTE può essere riutilizzata più volte nella stessa query (a differenza delle subquery che vanno riscritte ogni volta).
Gerarchia	Puoi definire più CTE concatenate separandole con una virgola.
Ricorsione	Le CTE possono essere ricorsive , cioè richiamare sé stesse (utile per alberi gerarchici come organigrammi o categorie).

Esempio con nomi di colonne personalizzati

```
WITH LotrHobbitsCTE (HobbitName, HobbitSurname) AS (
    SELECT nome, cognome
    FROM lotr.Hobbits
)
SELECT HobbitName, HobbitSurname
FROM LotrHobbitsCTE;
```

Qui vengono **rinominate le colonne** restituite dalla query interna.

Attenzione a ORDER BY nelle CTE

Le **tabelle in SQL non garantiscono mai un ordine fisso** dei record, e lo stesso vale per le **CTE**.

Pertanto:

- Inserire un ORDER BY **dentro la CTE** non è consigliato (e spesso non permesso, salvo con TOP).
- È **buona prassi** eseguire l'ordinamento **solo nel SELECT finale**.

Esempio corretto:

```
WITH EmployeeCTE AS (
    SELECT EmployeeID, Name, Salary
    FROM HumanResources.Employee
)
SELECT *
FROM EmployeeCTE
ORDER BY Salary DESC;
```

Ordinare tramite numero di colonna

Puoi ordinare specificando il **numero della colonna** indicato nel SELECT, utile in query rapide:

```
SELECT nome, cognome
FROM lotr.Hobbits
ORDER BY 1 DESC;  -- Ordina in base alla prima colonna (nome)
```

È una scorciatoia: per leggibilità, è **consigliabile usare il nome della colonna** invece del numero.

Funzione IIF

La funzione **IIF** è una **forma abbreviata del costrutto CASE WHEN**, introdotta in T-SQL a partire da SQL Server 2012.

Serve per **valutare una condizione logica e restituire uno dei due valori possibili** in base al risultato (vero/falso).

Sintassi

IIF(condizione, valore_se_vero, valore_se_falso)

Esempio pratico

```
SELECT
    OrderID,
    Quantity,
    IIF(Quantity > 10, 'MORE', 'LESS') AS QuantityStatus
FROM OrderDetails;
```

Spiegazione:

- Se Quantity è **maggiore di 10**, il risultato sarà 'MORE'.
- Se Quantity è **minore o uguale a 10**, il risultato sarà 'LESS'.

Quando usare IIF vs CASE WHEN

Situazione	Soluzione migliore
Solo due condizioni (vero/falso)	IIF
Più di due condizioni o logiche complesse	CASE WHEN
Controllo di valori NULL	Meglio usare CASE WHEN, più chiaro e prevedibile
Necessità di leggibilità	CASE WHEN è preferito in query lunghe

Esempio combinato

```
SELECT
    EmployeeID,
    IIF(Salary > 3000, 'High', 'Low') AS SalaryLevel,
    CASE
        WHEN Bonus > 1000 THEN 'Premium Bonus'
        WHEN Bonus BETWEEN 500 AND 1000 THEN 'Standard Bonus'
        ELSE 'No Bonus'
    END AS BonusCategory
FROM HumanResources.Employee;
```

In questo esempio:

- IIF valuta una condizione semplice (stipendio alto/basso).
- CASE valuta più condizioni per classificare i bonus.

T-SQL – Raggruppamenti, COUNT, GROUP BY e HAVING

COUNT e Distinzione tra COUNT(*) e COUNT(DISTINCT ...)

La funzione **COUNT()** viene utilizzata per **contare il numero di record** restituiti da una query. A seconda di come la usiamo, può **contare tutti i record** oppure **solo i valori unici**.

Contare tutti gli elementi

```
SELECT COUNT(*) AS TotaleRecord  
FROM lotr.Hobbits;
```

COUNT(*) conta **tutti i record** della tabella, **inclusi quelli con valori NULL**.

Nota:

COUNT(*) non valuta le colonne, ma **conta le righe**.

È il modo più efficiente per sapere **quanti record** ci sono in una tabella.



Contare valori non nulli di una colonna specifica

```
SELECT COUNT(Cognome) AS TotaleCognomi  
FROM lotr.Hobbits;
```

👉 COUNT(colonna) conta solo i record in cui **la colonna non è NULL**.

Contare valori distinti

```
SELECT COUNT(DISTINCT Cognome) AS CognomiUnici  
FROM lotr.Hobbits;
```

COUNT(DISTINCT colonna) conta **solo le occorrenze uniche**, escludendo i duplicati.

Attenzione:

Puoi usare DISTINCT solo su **una singola colonna** dentro COUNT() (non su più colonne contemporaneamente, a meno di concatenarle).

GROUP BY – Raggruppamento dei dati

L'istruzione **GROUP BY** permette di **raggruppare i dati in base a uno o più campi**, applicando su ciascun gruppo **funzioni di aggregazione** come:

- COUNT() – conta i record per gruppo
- SUM() – somma dei valori
- AVG() – media
- MIN() – valore minimo
- MAX() – valore massimo

Sintassi base

```
SELECT ColonnaRaggruppata, FunzioneAggregazione(Colonna)
FROM NomeTabella
GROUP BY ColonnaRaggruppata;
```

Esempio pratico

```
SELECT JobTitle, COUNT(*) AS NumeroDipendenti
FROM HumanResources.Employee
GROUP BY JobTitle;
```

Spiegazione:

- Raggruppa tutti i dipendenti per **JobTitle (ruolo)**.
- Per ogni gruppo (ruolo), **calcola il numero di dipendenti**.

Regola fondamentale del GROUP BY

Tutti i campi presenti nella SELECT devono essere:

- o inclusi nella clausola GROUP BY,
- oppure utilizzati all'interno di una funzione di aggregazione.

Se non rispetti questa regola, SQL Server restituirà un errore come:

Column 'NomeColonna' is invalid in the select list because it is not contained in either an aggregate function or the GROUP BY clause.

Esempio corretto

```
SELECT JobTitle, AVG(Salary) AS MediaStipendio
FROM HumanResources.Employee
GROUP BY JobTitle;
```

Esempio errato

```
SELECT JobTitle, Salary
FROM HumanResources.Employee
GROUP BY JobTitle;
```

Errore: Salary non è aggregata né inclusa nel GROUP BY.

HAVING – Filtrare dopo il raggruppamento

La clausola **HAVING** serve per **filtrare i risultati dopo un raggruppamento** (GROUP BY), mentre **WHERE** filtra **prima** del raggruppamento.

Differenza tra WHERE e HAVING

Clausola	Quando viene applicata	Su cosa agisce
WHERE	Prima del raggruppamento	Su singoli record
HAVING	Dopo il raggruppamento	Su gruppi di record

Esempio pratico

```
SELECT JobTitle, COUNT(*) AS NumeroDipendenti
FROM HumanResources.Employee
GROUP BY JobTitle
HAVING COUNT(*) > 5;
```

Spiegazione:

- Il **GROUP BY** crea gruppi per ogni tipo di JobTitle.
- **HAVING** filtra solo i gruppi con **più di 5 dipendenti**.

Esempio combinato con WHERE e HAVING

```
SELECT DepartmentID, AVG(Salary) AS MediaStipendio
FROM HumanResources.Employee
WHERE HireDate > '2015-01-01' -- Filtra PRIMA del gruppo
GROUP BY DepartmentID
HAVING AVG(Salary) > 3000;      -- Filtra DOPO il gruppo
```

Suggerimento:

- Usa **WHERE** per limitare i record iniziali.
- Usa **HAVING** per filtrare i risultati aggregati.

Esempi riassuntivi

1-Conteggio generale

```
SELECT COUNT(*) AS NumeroTotale
FROM lotr.Hobbits;
```

2-Conteggio univoco

```
SELECT COUNT(DISTINCT Cognome) AS CognomiDiversi
FROM lotr.Hobbits;
```

3-Raggruppamento

```
SELECT Cognome, COUNT(*) AS NumeroHobbits
FROM lotr.Hobbits
GROUP BY Cognome;
```

4-Raggruppamento con filtro HAVING

```
SELECT Cognome, COUNT(*) AS NumeroHobbits
FROM lotr.Hobbits
GROUP BY Cognome
HAVING COUNT(*) > 1;
```

Mostra solo i cognomi **che appaiono più di una volta**.

Ordine di esecuzione, Funzioni di Raggruppamento e Esercizi

Ordine di esecuzione dei comandi in una query SQL

Quando SQL Server elabora una query, **non esegue le clausole nell'ordine in cui le scriviamo**, ma segue un **ordine logico interno**.

Capire questo ordine è fondamentale per scrivere query corrette e prevederne il risultato.

Ordine logico di esecuzione

Ordine	Descrizione
1 FROM	SQL individua la tabella o le tabelle da cui leggere i dati. Qui possono già avvenire join, subquery , e CROSS/OUTER APPLY .
2 WHERE	Filtra i record eliminando le righe che non soddisfano la condizione . Agisce prima del raggruppamento .
3 GROUP BY	Raggruppa le righe rimanenti in base a uno o più campi. Serve per poter poi applicare funzioni di aggregazione .
5 HAVING	Filtra i gruppi creati dal GROUP BY in base a condizioni sulle funzioni di aggregazione .
6 SELECT	Seleziona le colonne o i valori da restituire. Possono essere colonne semplici o risultati di funzioni di aggregazione .

7 ORDER BY

Ordina i risultati finali secondo uno o più campi. È l'**ultimo passo** della query.

Schema riassuntivo

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

Nota importante:

SELECT viene scritto come prima clausola ma è **eseguito quasi per ultimo**.

Per questo motivo, non puoi usare in WHERE un alias definito nel SELECT, perché **non esiste ancora** a quel punto dell'esecuzione.

Esercizio 1 — Raggruppamento e filtro con OR

Richiesta:

Seleziona il **cognome** e il **conteggio** degli Hobbit che hanno cognome **“Baggins”** o **“Took”**, ordinando i risultati per cognome in **ordine decrescente**.

Soluzione:

```
SELECT
    Cognome,
    COUNT(*) AS NumeroHobbits
FROM lotr.Hobbits
WHERE Cognome IN ('Baggins', 'Took')
GROUP BY Cognome
ORDER BY Cognome DESC;
```

Spiegazione passo per passo:

1. **FROM:** prendo la tabella `lotr.Hobbits`.
2. **WHERE:** filtro solo i cognomi *Baggins* o *Took*.
3. **GROUP BY:** raggruppo per cognome.
4. **SELECT:** mostro cognome e conteggio.
5. **ORDER BY:** ordino in modo **decrescente** (da Z ad A).

Funzioni di raggruppamento

Le **funzioni di aggregazione** vengono utilizzate per calcolare valori riepilogativi (aggregati) su gruppi di righe.

Le principali sono:

Funzione	Descrizione
COUNT()	Conta il numero di righe o di valori non nulli.
SUM()	Somma i valori numerici di una colonna.
AVG()	Calcola la media dei valori numerici.
MIN()	Restituisce il valore minimo.
MAX()	Restituisce il valore massimo.

Esempio base

```
SELECT Cognome, MIN(Nome) AS PrimoNome
FROM Iotr.Hobbits
GROUP BY Cognome;
```

Spiegazione:

- Raggruppa tutti gli Hobbit con lo stesso cognome.
- Per ogni gruppo, restituisce il **primo nome alfabeticamente** (grazie a `MIN()`).

Regola chiave

Puoi **escludere la clausola GROUP BY** solo se **tutte le colonne** nella SELECT sono sottoposte a **funzioni di aggregazione**.

Esempio valido:

```
SELECT COUNT(*) AS NumeroTotale
FROM Iotr.Hobbits;
```

Esempio non valido:

```
SELECT Cognome, COUNT(*)  
FROM lotr.Hobbits;  
-- Errore: Cognome non è né aggregato né nel GROUP BY
```

Cosa mettere nel GROUP BY

Nel GROUP BY devi inserire **tutte le colonne** che compaiono nella SELECT **non coinvolte in funzioni di aggregazione**.

Esempio:

```
SELECT Cognome, COUNT(*) AS NumeroHobbits  
FROM lotr.Hobbits  
GROUP BY Cognome;
```

Qui Cognome non è aggregato, quindi **va nel GROUP BY**.

Esercizio 2 — Condizioni complesse

Richiesta:

Seleziona il **cognome** e il **conteggio** degli Hobbit che:

- hanno un cognome **compreso tra 'Baggins' e 'Took'**,
oppure
- hanno un nome che **inizia per 'D'**.

Ordina i risultati per **cognome crescente**.

Soluzione:

```
SELECT  
    Cognome,  
    COUNT(*) AS NumeroHobbits  
FROM lotr.Hobbits  
WHERE (Cognome BETWEEN 'Baggins' AND 'Took')  
    OR (Nome LIKE 'D%')  
GROUP BY Cognome
```

ORDER BY Cognome ASC;

Spiegazione:

- BETWEEN considera **tutti i cognomi in ordine alfabetico compresi tra Baggins e Took**.
- LIKE 'D%' seleziona tutti i nomi che **iniziano con la lettera D**.
- I risultati vengono **raggruppati per cognome** e **ordinati in ordine crescente**.

Esercizio 3 — CTE per cognomi duplicati

Richiesta:

Seleziona il **cognome** e il **nome** degli Hobbit che hanno un **cognome ripetuto più di una volta**.

Soluzione con CTE (Common Table Expression):

```
WITH CognomiDuplicati AS (  
    SELECT Cognome  
    FROM lotr.Hobbits  
    GROUP BY Cognome  
    HAVING COUNT(*) > 1  
)  
SELECT h.Cognome, h.Nome  
FROM lotr.Hobbits AS h  
INNER JOIN CognomiDuplicati AS cd  
    ON h.Cognome = cd.Cognome;
```

Spiegazione dettagliata:

1. **CTE (WITH):** crea un insieme temporaneo (CognomiDuplicati) che contiene **solo i cognomi ripetuti** più di una volta.
2. **HAVING COUNT(*) > 1:** filtra i cognomi che appaiono almeno due volte.
3. **SELECT finale:** prende nome e cognome da lotr.Hobbits, ma **solo** quelli che hanno un cognome presente nella CTE.

Vantaggio della CTE:

Rende la query **più leggibile e modulare**, evitando sottoquery annidate.

Tipi di dati, conversioni e gestione tabelle

Tipi di dati principali

In T-SQL, è fondamentale **conoscere i tipi di dati** perché influenzano la **memoria**, le **prestazioni** e la **precisione dei calcoli**.

Dati interi (Integer)

Tipo	Intervallo	Note
TINYINT	0 → 255	1 byte, ottimo per valori piccoli
SMALLINT	-32.768 → 32.767	2 byte, valori piccoli
INT	-2.147.483.648 → 2.147.483.647	4 byte, più comune
BIGINT	-9.223.372.036.854.775.808 → 9.223.372.036.854.775.807	8 byte, valori molto grandi

Suggerimento: usare il tipo più piccolo possibile per **risparmiare spazio e aumentare le prestazioni**.

Dati decimali / numerici

Tipo	Sintassi	Descrizione
NUMERIC(p,s)	p = precisione totale, s = cifre decimali	Esempio: NUMERIC(10,2) → 10 cifre totali, 2 decimali
DECIMAL(p,s)	Uguale a NUMERIC	Precisione e scala specificate

Dati booleani

Tipo	Valori possibili
BIT	0 o 1

Dati di tipo carattere

Tipo	Lunghezza	Note
CHAR(n)	fissa	Riempie con spazi se il contenuto è più corto, solo ASCII. Ideale per codici o nomi container.
VARCHAR(n)	variabile	Occupi solo lo spazio necessario fino a n. Può usare MAX per valori molto lunghi.
NCHAR(n)	fissa, Unicode	Permette caratteri internazionali.
NVARCHAR(n)	variabile, Unicode	Occupi solo lo spazio necessario, supporta più lingue.

Nota: NVARCHAR(MAX) può contenere fino a 2GB di testo.

Dati temporali

Tipo	Note
DATE	Solo data (anno-mese-giorno)
TIME	Solo orario
DATETIME	Data + ora, precisione limitata
DATETIME2	Data + ora, maggiore precisione
SMALLDATETIME	Data + ora, precisione minore, meno spazio

Funzioni di conversione dei tipi di dati

Spesso è necessario **convertire un tipo di dato in un altro**. T-SQL offre diverse funzioni.

CAST

```
SELECT CAST('123' AS INT) AS NumeroIntero;
```

- Converte un valore in un tipo specificato.
- Se la conversione non è valida (es. parola → INT), genera errore.

TRY_CAST

```
SELECT TRY_CAST('123' AS INT) AS NumeroIntero;
```

- Funziona come CAST ma **non genera errore** se la conversione fallisce.
- Restituisce NULL in caso di fallimento.

CONVERT

```
SELECT CONVERT(INT, '123') AS NumeroIntero;
```

- Conversione simile a CAST.
- Ha un parametro opzionale `style` per formati di **data/ora o stringhe**.

TRY_CONVERT

```
SELECT TRY_CONVERT(INT, 'ABC') AS NumeroIntero;
```

- Come TRY_CAST, restituisce NULL se la conversione non riesce.

Uso di VARCHAR(MAX)

```
DECLARE @Descrizione VARCHAR(MAX);
```

- Permette di contenere il **massimo numero di caratteri** possibile in T-SQL.
- Utile per testi molto lunghi come descrizioni o documenti.

Creazione di tabelle

```
CREATE TABLE lotr.Hobbits (  
    HobbitID INT PRIMARY KEY,  
    Nome NVARCHAR(50),
```

```
Cognome NVARCHAR(50),  
DataNascita DATE,  
IsActive BIT  
);
```

- Specificare sempre **il tipo di dato**.
- È buona pratica **usare schemi**, ad esempio `lotr`, per organizzare le tabelle.

Aggiornamento di tabelle (UPDATE)

```
UPDATE lotr.Hobbits  
SET IsActive = 0  
WHERE Cognome = 'Baggins';
```

Tre casi principali in modifiche concorrenti:

Quando più utenti accedono e modificano la stessa tabella, possono verificarsi:

- 1. Aggiornamento corretto**
 - a. Nessun conflitto, il valore viene aggiornato come previsto.
- 2. Aggiornamento fallito per lock**
 - a. Un altro utente sta modificando la stessa riga → SQL Server blocca l'operazione fino al rilascio.
- 3. Aggiornamento overwriting**
 - a. Due utenti aggiornano contemporaneamente la stessa riga senza controllo → l'ultima scrittura **sovrascrive** la precedente.

Consiglio: usare **transaction**(`BEGIN TRAN`, `COMMIT`, `ROLLBACK`) per gestire aggiornamenti concorrenti in sicurezza.

T-SQL – Livelli di isolamento, transazioni e linguaggio Microsoft

T-SQL: il linguaggio Microsoft

T-SQL (Transact-SQL) è l'estensione del linguaggio **SQL** utilizzata da **Microsoft SQL Server**. Oltre alle classiche istruzioni SQL standard, introduce **funzionalità aggiuntive** come:

- **Gestione delle transazioni**
- **Controllo del flusso** (IF, WHILE, TRY/CATCH)
- **Variabili e procedure**
- **Gestione dei livelli di isolamento**

T-SQL quindi **non è solo dichiarativo**, ma può assumere un comportamento **procedurale**, permettendo logiche complesse direttamente lato database.

Il concetto di transazione

Una **transazione** è un insieme di operazioni che devono essere eseguite **tutte insieme o nessuna**. In altre parole, segue le regole **ACID**:

Proprietà	Significato
Atomicity	Tutte le operazioni vengono completate o nessuna di esse viene applicata
Consistency	Il database passa da uno stato coerente a un altro
Isolation	Le transazioni non interferiscono tra loro
Durability	Una volta fatto il commit, i dati restano salvati anche in caso di crash

Esempio classico: il bonifico bancario

Supponiamo di voler trasferire **100€ da un conto A a un conto B**.

Questa operazione richiede due passi:

1. Sottrarre 100€ dal conto A
2. Aggiungere 100€ al conto B

Se la transazione si interrompe tra il primo e il secondo passo, il denaro “scompare”.

Per evitare ciò, usiamo una **transazione**:

```
BEGIN TRANSACTION;
```

```
UPDATE Conti  
SET Saldo = Saldo - 100  
WHERE NumeroConto = 'A';
```

```
UPDATE Conti  
SET Saldo = Saldo + 100  
WHERE NumeroConto = 'B';
```

```
COMMIT TRANSACTION;
```

Se qualcosa va storto (ad esempio, il secondo update fallisce), possiamo **annullare le modifiche**:

```
ROLLBACK TRANSACTION;
```

Importante: se si verifica un errore e non si esegue il ROLLBACK, la transazione rimane **aperta** e può bloccare altre operazioni.

Esempio di transazione non andata a buon fine

Se durante l'operazione qualcosa fallisce (es. connessione persa), potremmo vedere **temporaneamente** che i soldi sono stati sottratti ma non ancora aggiunti all'altro conto.

Questo accade se qualcuno legge la tabella con un livello di isolamento **meno restrittivo**, ad esempio con **NOLOCK** o **READ UNCOMMITTED**, che permettono la lettura di dati **non ancora committati**.

Livelli di isolamento

I **livelli di isolamento** determinano **come le transazioni vedono e bloccano i dati**.

Essi definiscono la “visibilità” delle modifiche durante l'esecuzione concorrente di più operazioni.

1- READ UNCOMMITTED

- Permette di leggere anche dati **non ancora committati** (cioè modifiche temporanee).
- **Massima velocità**, ma **rischio di dati sporchi (dirty reads)**.
- Equivalente all'uso dell'hint **WITH (NOLOCK)**.

```
SELECT * FROM Conti WITH (NOLOCK);
```

Rischio: potresti leggere un valore che viene subito dopo annullato da un rollback.

2-READ COMMITTED (default)

- È il **livello di isolamento predefinito** in SQL Server.
- Permette di leggere **solo dati che sono stati committati**.
- Durante una scrittura, le righe coinvolte vengono **bloccate** fino al commit.

Vantaggio: dati sempre coerenti

Svantaggio: le letture possono **attendere** se un'altra transazione ha un lock attivo.

3-REPEATABLE READ

- Una volta letti i dati, **nessun'altra transazione può modificarli** finché la tua non termina.
- Impedisce le **non-repeatable reads** (cioè il valore che cambia tra due letture nella stessa transazione).
- Blocca le righe lette.

4-SNAPSHOT

- SQL Server fa una **fotografia (snapshot)** dei dati nel momento in cui la transazione inizia.
- Anche se altri utenti modificano i dati, tu **vedi sempre la versione originale**.
- Nessun blocco di lettura: le transazioni non si bloccano a vicenda.

Ottimo per sistemi con molte letture concorrenti.

Usa più spazio nel **tempdb** per mantenere le versioni dei dati.

5-SERIALIZABLE

- Livello più alto di isolamento.
- Le transazioni vengono eseguite **una per volta**, come se fossero in sequenza.
- Impedisce **tutti i fenomeni di concorrenza** (dirty, non-repeatable e phantom reads).
- Blocca interi range di righe.

Molto sicuro, ma riduce fortemente le prestazioni.

Riepilogo livelli di isolamento

Livello	Legge dati non committati	Previene modifiche durante la lettura	Blocchi lunghi	Uso consigliato
READ UNCOMMITTED	Sì	No	Nessuno	Solo per report o statistiche
READ COMMITTED	No	No	Medio	Default, bilanciato
REPEATABLE READ	No	Sì	Alto	Transazioni critiche
SNAPSHOT	No	Sì	Nessuno	Lecture parallele sicure
SERIALIZABLE	No	Sì	Molto alto	Massima coerenza

Il ruolo di NOLOCK

L'hint **WITH (NOLOCK)** viene usato per **aggirare i lock di lettura** nel livello **READ COMMITTED**.

Permette di **leggere dati anche se non committati**, migliorando le prestazioni ma sacrificando la consistenza.

```
SELECT * FROM HumanResources.Employee WITH (NOLOCK);
```

Attenzione: puoi leggere dati **inconsistenti**, o addirittura righe “fantasma” che vengono poi eliminate da un rollback.

Operazioni che richiedono DROP e ricreazione della tabella

Alcune modifiche **non possono essere fatte direttamente** con un **ALTER TABLE**, ma richiedono la **ricreazione della tabella**.

Un esempio è **rimuovere il vincolo NOT NULL** da una colonna.

-- Operazione non consentita direttamente:

```
ALTER TABLE Clienti
```

```
ALTER COLUMN Nome VARCHAR(50) NULL;
```

In alcuni casi, SQL Server impone di:

1. Creare una **nuova tabella** con la struttura modificata,
2. Copiare i dati,
3. Eliminare la tabella originale,

4. Rinominare la nuova tabella.

GROUP BY, HAVING e gestione dei valori NULL

GROUP BY: concetto generale

L'istruzione **GROUP BY** serve per **raggruppare i dati** in base a uno o più campi e applicare **funzioni di aggregazione** (come SUM, COUNT, AVG, MIN, MAX) su ciascun gruppo.

Esempio base:

```
SELECT Cognome, SUM(Eta) AS SommaEta
FROM lotr.hobbits
GROUP BY Cognome;
```

Questo comando restituisce la **somma dell'età** per ciascun cognome (famiglia).

Inclusione dei valori NULL

Per impostazione predefinita, il **GROUP BY** **include anche i valori NULL** come un gruppo separato. Tuttavia, quando si contano i valori, bisogna fare attenzione:

- **COUNT(colonna)** **non conta** i valori NULL.
- **COUNT(*)** o **COUNT(1)** **contano tutte le righe**, anche se contengono NULL.

Trucco utile:

Se voglio includere anche i record con valori NULL nella mia statistica, posso usare **COUNT(1)** o **COUNT(*)**.

Esempio:

```
SELECT
    Cognome AS Famiglia,
    SUM(Eta) AS SommaEta,
    COUNT(1) AS NumeroComponenti
FROM lotr.hobbits
GROUP BY Cognome;
```

Anche se Cognome fosse NULL, quel gruppo verrà incluso e contato.

HAVING: filtro sui gruppi

Mentre **WHERE** filtra **le righe** prima del raggruppamento, **HAVING** filtra **i gruppi dopo il raggruppamento**.

Esempio 1 – Somma di età superiore a 50

“Nome di tutte le famiglie che hanno somma di età > 50”

```
SELECT Cognome, SUM(Eta) AS SommaEta
FROM lotr.hobbits
GROUP BY Cognome
HAVING SUM(Eta) > 50;
```

Spiegazione:

- GROUP BY Cognome: raggruppa per famiglia.
- SUM(Eta): calcola la somma delle età in ogni famiglia.
- HAVING SUM(Eta) > 50: mostra solo le famiglie con somma età > 50.

Filtrare i gruppi in base a condizioni testuali

“Tutte le famiglie che iniziano con la lettera B”

Puoi aggiungere una condizione WHERE **prima del raggruppamento**, così:

```
SELECT Cognome, SUM(Eta) AS SommaEta
FROM lotr.hobbits
WHERE Cognome LIKE 'B%'
GROUP BY Cognome;
```

- WHERE filtra **le righe**, non i gruppi.
- HAVING filtra **i gruppi aggregati**.

Raggruppare per più colonne

“Voglio avere la somma delle età dei componenti maschi e separatamente delle componenti femmine di ciascuna famiglia.”

Serve raggruppare **per più attributi**:

```
SELECT
    Cognome AS Famiglia,
    Sesso,
    SUM(Eta) AS SommaEta
FROM lotr.hobbits
GROUP BY Cognome, Sesso
ORDER BY Cognome, Sesso;
```

Risultato:

- Ogni famiglia (Cognome) comparirà due volte: una per M e una per F.
- SUM(Eta) calcola la somma separata per ciascun sesso.

Regole fondamentali per l'uso di GROUP BY

Le colonne non aggregate devono comparire nel GROUP BY

Ogni colonna presente nella SELECT che **non** è in una funzione di aggregazione (SUM, AVG, ecc.) **deve obbligatoriamente** essere elencata anche nel GROUP BY.

Esempio corretto :

```
SELECT Cognome, SUM(Eta)
FROM lotr.hobbits
GROUP BY Cognome;
```

Esempio errato :

```
SELECT Cognome, Nome, SUM(Eta)
FROM lotr.hobbits
GROUP BY Cognome;
Errore: la colonna Nome non è né aggregata né nel GROUP BY.
```

Non è possibile fare ORDER BY su colonne non presenti nel SELECT o nel GROUP BY

Se si prova a ordinare per un campo **non incluso nel risultato né nel gruppo**, SQL Server restituisce un errore.

Esempio errato:

```
SELECT Cognome, SUM(Eta)
FROM lotr.hobbits
GROUP BY Cognome
ORDER BY Eta; -- Errore
```

Esempio corretto:

```
SELECT Cognome, SUM(Eta) AS SommaEta
FROM lotr.hobbits
GROUP BY Cognome
ORDER BY SommaEta DESC;
```

In SELECT puoi avere colonne non presenti nel GROUP BY solo se aggregate

Puoi selezionare colonne non presenti nel GROUP BY **solo** se le stai aggregando (con SUM, AVG, COUNT, ecc.).

Il contrario non vale: puoi avere colonne nel GROUP BY che **non sono** nella SELECT.

Esempio:

```
SELECT Cognome, SUM(Eta)
FROM lotr.hobbits
GROUP BY Cognome, Sesso; -- 'Sesso' non compare nella SELECT, ma è ammesso
```

Chiave primaria (PRIMARY KEY)

Definizione

Una **chiave primaria (Primary Key)** è **una colonna o un insieme di colonne** in una tabella che **identificano in modo univoco ogni record** (riga) della tabella.

In altre parole, **nessun valore nella chiave primaria può essere duplicato o NULL**.
Ogni record deve poter essere distinto da tutti gli altri attraverso la chiave primaria.

Esempio:

```
CREATE TABLE lotr.Hobbits (
    HobbitID INT PRIMARY KEY,
    Nome NVARCHAR(50),
    Cognome NVARCHAR(50)
);
```

In questo esempio, HobbitID è la chiave primaria e garantisce che **ogni hobbit abbia un identificativo unico**.

Caratteristiche fondamentali della chiave primaria

- 1. Unicità:**
I valori della chiave primaria **devono essere unici**. Nessun record può avere la stessa chiave primaria di un altro.
- 2. Non nullabilità:**
I campi che costituiscono una chiave primaria **non possono contenere valori NULL**.
- 3. Indicizzazione automatica:**
Quando si crea una chiave primaria, SQL Server crea **automaticamente un indice cluster** (clustered index) su di essa, se non ne esiste già uno.
Questo **ottimizza le ricerche** e gli **ordinamenti** basati su quella colonna.
- 4. Unicità logica:**
La chiave primaria serve a **identificare logicamente il record**, indipendentemente dal contenuto delle altre colonne.

Tipologie di chiave primaria

Chiave naturale

È una chiave primaria composta da uno o più campi **già esistenti** nella realtà del dominio, ad esempio:

- il codice fiscale per una persona,
- la targa per un'auto,
- il codice ISBN per un libro.

Esempio:

```
CREATE TABLE Persone (  
    CodiceFiscale CHAR(16) PRIMARY KEY,  
    Nome NVARCHAR(50),  
    Cognome NVARCHAR(50)  
);
```

Vantaggio: rappresenta un'informazione reale e significativa.

Svantaggio: se cambia (evento raro ma possibile), si deve aggiornare in tutte le tabelle collegate.

Chiave surrogata

È una chiave **artificiale** creata **appositamente** per identificare i record.

Generalmente si tratta di un campo numerico incrementale (es. ID, Codice, RecordNumber).

Esempio:

```
CREATE TABLE Clienti (  
    ClienteID INT IDENTITY(1,1) PRIMARY KEY,  
    Nome NVARCHAR(50),  
    Cognome NVARCHAR(50)
```


);

Spiegazione:

- La parola chiave `IDENTITY(1,1)` fa sì che SQL Server assegni **automaticamente valori progressivi** alla colonna (1, 2, 3, ...).
- Ogni record inserito avrà un **ID univoco**, indipendente dal contenuto dei campi reali.

Vantaggi della chiave surrogata:

- Non cambia mai (stabilità nel tempo).
- È **efficiente** da confrontare e indicizzare.
- Evita di dipendere da dati reali (che potrebbero cambiare).

Svantaggio: non ha significato nel mondo reale, serve solo per il database.

Chiavi primarie composte

Una **chiave primaria composta** è una chiave che utilizza **più colonne insieme** per garantire l'unicità.

Esempio:

```
CREATE TABLE Partecipazioni (  
    EventoID INT,  
    PartecipanteID INT,  
    PRIMARY KEY (EventoID, PartecipanteID)  
);
```

In questo caso, nessuno dei due campi da solo è univoco, ma la **combinazione dei due** lo è (ogni partecipante può partecipare più eventi, ma non due volte allo stesso evento).

Buone pratiche nell'uso delle chiavi primarie

1. **Preferisci chiavi surroghe per tabelle di grandi dimensioni o soggette a variazioni.**
(Evita chiavi naturali che potrebbero cambiare, come email o numero di telefono.)
2. **Usa chiavi composte solo se necessario**, ad esempio per tabelle di relazione (molti-a-molti).
3. **Nomina chiaramente la chiave**, ad esempio `ClienteID`, `OrdineID`, `LibroID`.
4. **Evita chiavi troppo lunghe o con tipi di dato pesanti** (come `NVARCHAR(255)`).

JOIN e UNION

Cos'è una JOIN

Una **JOIN** serve per **mettere in relazione due o più tabelle** in base a una condizione di corrispondenza (di solito fra chiavi primarie e chiavi esterne).

Permette di **combinare dati** provenienti da più tabelle come se fossero un'unica sorgente.

Quando lavoriamo con query da più tabelle è consigliato usare gli alias per rendere il codice più leggibile.

Tipi di JOIN

INNER JOIN

Restituisce **solo i record che hanno corrispondenza in entrambe le tabelle**.

```
SELECT f.Titolo, f.AnnoDiUscita, l.AnnoDiUscita
FROM Film AS f
INNER JOIN Libri AS l
    ON f.Titolo = l.Titolo;
```

Mostra solo i titoli che **esistono sia come film che come libro**.

LEFT JOIN

Restituisce **tutti i record della tabella di sinistra**, e solo quelli corrispondenti della tabella di destra (se presenti).

Se non c'è corrispondenza, i valori della tabella di destra saranno **NULL**.

```
SELECT f.Titolo, f.AnnoDiUscita, l.AnnoDiUscita
FROM Film AS f
LEFT JOIN Libri AS l
    ON f.Titolo = l.Titolo;
```

Con un **LEFT JOIN sui film**, siamo certi che **tutti i titoli dei film saranno valorizzati**, anche se non hanno un libro corrispondente.

RIGHT JOIN

È l'opposto del LEFT JOIN: restituisce **tutti i record della tabella di destra** e solo quelli corrispondenti della sinistra.

```
SELECT f.Titolo, f.AnnoDiUscita, l.AnnoDiUscita
FROM Film AS f
RIGHT JOIN Libri AS l
```

```
ON f.Titolo = l.Titolo;
```

Qui otterremo **tutti i libri**, anche quelli che non hanno un film associato.

FULL JOIN (FULL OUTER JOIN)

Restituisce **tutti i record di entrambe le tabelle**, con valori NULL dove non ci sono corrispondenze.

```
SELECT f.Titolo, f.AnnoDiUscita AS AnnoFilm, l.AnnoDiUscita AS AnnoLibro
FROM Film AS f
FULL OUTER JOIN Libri AS l
    ON f.Titolo = l.Titolo;
```

È utile per avere una **vista completa** di tutti i titoli, sia di film che di libri.

Esercizi ed esempi pratici

1. Trovare i titoli che hanno sia film che libro

```
SELECT f.Titolo
FROM Film AS f
INNER JOIN Libri AS l
    ON f.Titolo = l.Titolo;
```

Usa **INNER JOIN** per mostrare solo i titoli presenti in entrambe le tabelle.

2. Mostrare i titoli che sono solo film o solo libri

(ovvero se un titolo non esiste nell'altra tabella)

Soluzione con CASE WHEN:

```
SELECT
    CASE
        WHEN l.Titolo IS NULL THEN f.Titolo
        ELSE l.Titolo
    END AS Titolo
FROM Film AS f
FULL JOIN Libri AS l
    ON f.Titolo = l.Titolo
WHERE f.Titolo IS NULL OR l.Titolo IS NULL;
```

Soluzione con ISNULL o IIF:

```
SELECT ISNULL(f.Titolo, l.Titolo) AS Titolo
FROM Film AS f
FULL JOIN Libri AS l
    ON f.Titolo = l.Titolo
WHERE f.Titolo IS NULL OR l.Titolo IS NULL;
```

Queste query **escludono i titoli** che hanno entrambe le versioni e mostrano solo quelli **unici** a una delle due tabelle.

Conversioni di tipo (CAST)

Quando si devono confrontare colonne di tipi diversi (es. int e nvarchar), si usa il **CAST** o il **CONVERT**.

```
SELECT f.Titolo, CAST(f.AnnoDiUscita AS NVARCHAR(4)) AS AnnoStringa
FROM Film AS f;
```

È fondamentale per confrontare o concatenare colonne di tipo diverso.

UNION

La **UNION** permette di **combinare i risultati di due o più SELECT** in un'unica tabella temporanea, a patto che:

- Le colonne selezionate abbiano **lo stesso numero e tipo di dati compatibili**;
- I nomi delle colonne sono presi dalla **prima SELECT**.
- Fa automaticamente una distinct

```
SELECT Titolo, AnnoDiUscita
FROM Film
UNION
SELECT Titolo, AnnoDiUscita
FROM Libri;
```

Rimuove automaticamente i duplicati.

Se vuoi mantenerli, usa **UNION ALL**.

OUTER JOIN e Query Avanzate

Cos'è una OUTER JOIN

Una **OUTER JOIN** è una **variante della JOIN** che restituisce **tutti i record di una o entrambe le tabelle**, anche se **non esiste una corrispondenza** tra le chiavi di collegamento.

È il contrario di una **INNER JOIN**, che mostra solo i record che combaciano.

Tipi di OUTER JOIN

LEFT OUTER JOIN

Restituisce **tutti i record della tabella sinistra** e i dati corrispondenti della tabella destra (se esistono).

Dove non ci sono corrispondenze, i campi della tabella destra vengono impostati su **NULL**.

```
SELECT *  
FROM Film AS f  
LEFT JOIN Libri AS l  
    ON f.Titolo = l.Titolo;
```

Usa un **LEFT JOIN** quando vuoi **conservare tutti i film**, anche se non hanno un libro corrispondente.

RIGHT OUTER JOIN

Restituisce **tutti i record della tabella destra**, più i dati corrispondenti della sinistra.

È meno usata perché si può ottenere lo stesso risultato invertendo l'ordine delle tabelle con un **LEFT JOIN**.

```
SELECT *  
FROM Film AS f  
RIGHT JOIN Libri AS l  
    ON f.Titolo = l.Titolo;
```

FULL OUTER JOIN

Restituisce **tutti i record di entrambe le tabelle**, con valori NULL dove non ci sono corrispondenze.

È utile per confrontare set di dati e trovare **corrispondenze, esclusioni o mancanze**.

```
SELECT *  
FROM Film AS f  
FULL OUTER JOIN Libri AS l
```

ON f.Titolo = l.Titolo;

Esercizio: Prodotto Cartesiano – “Battaglia Navale”

Il **prodotto cartesiano** è una particolare operazione ottenuta con una **CROSS JOIN** o con una **JOIN** **senza condizione ON**.

Restituisce **tutte le combinazioni possibili** tra le righe delle due tabelle.

Esempio pratico — “Battaglia Navale”:

Supponiamo di avere due tabelle:

- **Righe** con valori da 1 a 10
- **Colonne** con lettere da A a J

```
SELECT R.NomeRiga, C.NomeColonna
FROM Righe AS R
CROSS JOIN Colonne AS C;
```

Il risultato sarà una **griglia 10x10** con tutte le coordinate (es. A1, B1, C1, ...), come nel gioco della **Battaglia Navale**.

Questa tecnica è utile per **generare combinazioni** di dati, testare join o creare dati simulati.

Esercizi pratici con JOIN

1. Film tratti da libri usciti dopo il 2000

Obiettivo: trovare **solo i film** che sono stati tratti da **libri pubblicati dopo il 2000**.

```
SELECT f.Titolo, f.AnnoDiUscita, l.AnnoDiUscita AS AnnoLibro
FROM Film AS f
INNER JOIN Libri AS l
    ON f.Titolo = l.Titolo
WHERE l.AnnoDiUscita > 2000;
```

Qui usiamo **INNER JOIN** perché vogliamo **solo le corrispondenze** tra film e libri.
La condizione `WHERE l.AnnoDiUscita > 2000` filtra i soli libri recenti.

2. Elenco di tutti i libri e l'anno del film solo se il film è del 1941

Richiesta: mostra **tutti i libri**, e nella **terza colonna** scrivi **l'anno del film** solo **se il film è uscito nel 1941**, altrimenti lascia NULL.

Soluzione 1 — con LEFT JOIN e IIF

```
SELECT
    l.Titolo,
    l.AnnoDiUscita AS AnnoLibro,
    IIF(f.AnnoDiUscita = 1941, f.AnnoDiUscita, NULL) AS AnnoFilm
FROM Libri AS l
LEFT JOIN Film AS f
    ON l.Titolo = f.Titolo;
```

Soluzione 2 — con condizione aggiuntiva nella JOIN

```
SELECT
    l.Titolo,
    l.AnnoDiUscita AS AnnoLibro,
    f.AnnoDiUscita AS AnnoFilm
FROM Libri AS l
LEFT JOIN Film AS f
    ON l.Titolo = f.Titolo AND f.AnnoDiUscita = 1941;
```

Il **LEFT JOIN** garantisce che **tutti i libri** vengano visualizzati, anche se non esiste un film del 1941. La condizione nella **ON** limita l'associazione solo ai film di quell'anno.

3. Libri da cui è stato tratto un film, con classificazione per periodo storico

Obiettivo: mostrare **solo i libri che hanno ispirato un film**, e aggiungere una colonna che indica il periodo storico del libro:

Periodo	Etichetta
< 1500	“Vetusto”
1500–1799	“Antico”
1800–1899	“Vecchio”
1900–1999	“Vecchino ”
≥ 2000	“Nuovo”

```
SELECT
    l.Titolo,
    l.AnnoDiUscita,
    f.AnnoDiUscita AS AnnoFilm,
```

```

CASE
    WHEN l.AnnoDiUscita < 1500 THEN 'Vetusto'
    WHEN l.AnnoDiUscita BETWEEN 1500 AND 1799 THEN 'Antico'
    WHEN l.AnnoDiUscita BETWEEN 1800 AND 1899 THEN 'Vecchio'
    WHEN l.AnnoDiUscita BETWEEN 1900 AND 1999 THEN 'Vecchino'
    ELSE 'Nuovo'
END AS Periodo
FROM Libri AS l
INNER JOIN Film AS f
    ON l.Titolo = f.Titolo;

```

Usiamo **INNER JOIN** perché vogliamo **solo i libri che hanno un film corrispondente**, e la **CASE** per la categorizzazione temporale.

4. Conteggio dei film prodotti per anno

Obiettivo: sapere **quanti film** sono stati prodotti in ciascun anno.

```

SELECT
    AnnoDiUscita,
    COUNT(*) AS NumeroFilm
FROM Film
GROUP BY AnnoDiUscita
ORDER BY AnnoDiUscita;

```

Si utilizza la **clausola GROUP BY** per raggruppare per anno e **COUNT()** per contare i film. È un esempio classico di **aggregazione dati**.

Principi ACID nei Database Relazionali (T-SQL)

I **principi ACID** sono le **quattro proprietà fondamentali** che garantiscono l'affidabilità delle **transazioni nei database**.

Una **transazione** è un'unità logica di lavoro che può comprendere **una o più istruzioni SQL** (come INSERT, UPDATE, DELETE) da eseguire come un blocco indivisibile.

Una transazione deve garantire che **il database rimanga sempre in uno stato consistente**, anche in caso di errore o interruzione.

A → Atomicity (Atomicità)

Definizione:

L'**atomicità** assicura che **tutte le operazioni** di una transazione vengano eseguite **completamente o per niente**.

Se una parte fallisce, **l'intera transazione viene annullata** e il database ritorna allo stato precedente (rollback).

Esempio pratico – operazione bancaria:

Trasferire 50€ dal conto di *Tizio* a *Caio* deve essere **un'operazione unica e indivisibile**.

Non può accadere che i soldi vengano scalati da Tizio ma non accreditati a Caio.

```
INSERT INTO Log VALUES ('Apro operazione bonifico');
```

```
BEGIN TRANSACTION;
```

```
BEGIN TRY
```

```
    UPDATE Conto
```

```
    SET Saldo = Saldo - 50
```

```
    WHERE Persona = 'Tizio';
```

```
    UPDATE Conto
```

```
    SET Saldo = Saldo + 50
```

```
    WHERE Persona = 'Caio';
```

```
    COMMIT TRANSACTION; -- Tutto va bene → confermo
```

```
END TRY
```

```
BEGIN CATCH
```

```
    ROLLBACK TRANSACTION; -- Errore → annullo tutto
```

```
    INSERT INTO Log VALUES ('Errore durante il bonifico');
```

```
END CATCH;
```

In caso di errore, la transazione viene annullata con ROLLBACK.

Se tutto va bene, viene confermata con COMMIT.

L'operazione è **atomica** perché non può essere divisa: o avviene tutta, o non avviene affatto.

C → Consistency (Consistenza)

Definizione:

La **consistenza** garantisce che **ogni transazione porti il database da uno stato valido a un altro stato valido**.

In altre parole, **le regole di integrità (vincoli, chiavi, relazioni)** devono essere rispettate prima e dopo la transazione.

Esempio:

Se un trasferimento causa un saldo negativo e nel database c'è una **regola di vincolo CHECK** che vieta saldi sotto zero, la transazione **deve fallire**.

```
ALTER TABLE Conto  
ADD CONSTRAINT CK_Saldo CHECK (Saldo >= 0);
```

Se una UPDATE viola questa regola, SQL Server **rifiuta l'operazione** e il sistema mantiene la consistenza.

I → Isolation (Isolamento)

Definizione:

L'**isolamento** assicura che **le transazioni multiple in esecuzione simultanea non interferiscano tra loro**.

Ogni transazione deve comportarsi **come se fosse l'unica in esecuzione sul database**.

SQL Server garantisce diversi **livelli di isolamento**, come:

Livello	Descrizione
READ UNCOMMITTED	Legge anche dati non ancora confermati (dirty read)
READ COMMITTED (default)	Legge solo dati già confermati
REPEATABLE READ	Blocca le righe lette per evitare modifiche parallele
SERIALIZABLE	Massima protezione, blocca tutto il range di dati
SNAPSHOT	Usa versioni dei dati (senza blocchi)

Esempio:

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

```
BEGIN TRANSACTION;
```

```
SELECT * FROM Conto WHERE Persona = 'Tizio';
```

```
-- I dati letti da questa query non potranno essere modificati da altre transazioni
```

```
COMMIT;
```

D → Durability (Durabilità)

Definizione:

La **durabilità** assicura che, **una volta eseguito il COMMIT**, i dati rimangano salvati **in modo permanente**, anche in caso di crash del sistema, blackout o riavvio del server.

SQL Server garantisce questa proprietà tramite:

- Il **transaction log** (registro delle transazioni), che conserva tutte le operazioni finché non vengono scritte su disco.
- Meccanismi di **recovery** automatico all'avvio.

Esempio concettuale:

Dopo COMMIT, i dati vengono scritti nel log e poi nel file `.mdf` (database fisico).

Anche se si spegne la macchina, al riavvio SQL Server ripristina i dati dal log.

Riassunto

Principio	Nome	Significato	Esempio
A	Atomicity	Tutto o niente	Bonifico bancario completo
C	Consistency	Regole sempre rispettate	Saldo ≥ 0
I	Isolation	Le transazioni non interferiscono	Letture protette
D	Durability	Dati permanenti dopo COMMIT	Persistenza su disco

I principi ACID sono ciò che rende i **database relazionali affidabili**, soprattutto in **sistemi critici** (banche, sanità, e-commerce).

In T-SQL, questi principi vengono gestiti automaticamente dal **motore delle transazioni di SQL Server**, ma come sviluppatori dobbiamo:

- Scrivere **transazioni complete e coerenti**.
- Usare **TRY/CATCH** per gestire gli errori.
- Scegliere il **livello di isolamento** appropriato.
- Evitare operazioni che compromettano la **consistenza**.

Chiavi Esterne (Foreign Keys) in T-SQL

Definizione

Una **chiave esterna (foreign key)** è un vincolo di integrità referenziale che serve per **collegare due tabelle**.

Essa impone che i valori di una colonna (o gruppo di colonne) in una tabella **corrispondano ai valori di una chiave primaria** in un'altra tabella.

In pratica, le chiavi esterne **creano relazioni tra entità** (es. libri, autori, utenti, prestiti).

Quando usare una chiave esterna

Si usa una **foreign key** quando:

- si vuole **collegare due tabelle correlate** (es. ogni libro ha un autore);
- si vuole **impedire l'inserimento di dati incoerenti** (es. un libro con un autore inesistente);
- si vuole garantire **l'integrità referenziale**:
non è possibile cancellare o modificare un record se esistono record collegati che lo utilizzano.

Esempio 1 – Relazione 1 a molti (Autori → Libri)

In una relazione **uno a molti**, **un autore può scrivere più libri**, ma **ogni libro** può essere scritto **da un solo autore**.

Tabelle:

```
CREATE TABLE Autori (  
    IdAutore INT PRIMARY KEY IDENTITY(1,1),  
    Nome NVARCHAR(100) NOT NULL,  
    Cognome NVARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE Libri (  
    IdLibro INT PRIMARY KEY IDENTITY(1,1),  
    Titolo NVARCHAR(200) NOT NULL,  
    AnnoPubblicazione INT,  
    IdAutore INT NOT NULL, -- Chiave esterna  
  
    CONSTRAINT FK_Libri_Autori  
        FOREIGN KEY (IdAutore)  
        REFERENCES Autori(IdAutore)
```

```
ON UPDATE CASCADE
ON DELETE NO ACTION
```

```
);
```

Spiegazione:

- IdAutore in **Libri** è una **foreign key** che punta alla **primary key** di **Autori**.
- ON UPDATE CASCADE → se l'Id dell'autore cambia, si aggiorna anche nei libri.
- ON DELETE NO ACTION → non si può cancellare un autore che ha libri collegati.

Inserimento dati:

```
INSERT INTO Autori (Nome, Cognome)
VALUES ('Isaac', 'Asimov');
```

```
INSERT INTO Libri (Titolo, AnnoPubblicazione, IdAutore)
VALUES ('Fondazione', 1951, 1);
```

Esempio 2 – Relazione molti a molti (Autori ↔ Libri)

In alcuni casi, **un libro può avere più autori** e **un autore può aver scritto più libri**.

In questo scenario, non basta la chiave esterna diretta: serve una **tabella di collegamento** (o tabella ponte).

Struttura:

```
CREATE TABLE Autori (
    IdAutore INT PRIMARY KEY IDENTITY(1,1),
    Nome NVARCHAR(100)
);
```

```
CREATE TABLE Libri (
    IdLibro INT PRIMARY KEY IDENTITY(1,1),
    Titolo NVARCHAR(200)
);
```

```
CREATE TABLE Autori_Libri (
    IdAutore INT,
    IdLibro INT,
    PRIMARY KEY (IdAutore, IdLibro),

    CONSTRAINT FK_AutoriLibri_Autori
    FOREIGN KEY (IdAutore) REFERENCES Autori(IdAutore),
```

```
CONSTRAINT FK_AutoriLibri_Libri
    FOREIGN KEY (IdLibro) REFERENCES Libri(IdLibro)
);
```

La tabella **Autori_Libri** rappresenta la relazione **multi-a-molti**, contenendo **le chiavi primarie delle due tabelle principali**.

◇ Esempio 3 – Tabella di cross (Prestiti)

Una **tabella di cross (intermedia)** collega più entità in una relazione complessa.

Ad esempio, un sistema bibliotecario può gestire i **prestiti** con una tabella che collega **utenti** e **libri**.

```
CREATE TABLE Utenti (
    IdUtente INT PRIMARY KEY IDENTITY(1,1),
    Nome NVARCHAR(100),
    Cognome NVARCHAR(100)
);
```

```
CREATE TABLE Libri (
    IdLibro INT PRIMARY KEY IDENTITY(1,1),
    Titolo NVARCHAR(200)
);
```

```
CREATE TABLE Prestiti (
    IdPrestito INT PRIMARY KEY IDENTITY(1,1),
    IdUtente INT NOT NULL,
    IdLibro INT NOT NULL,
    DataPrestito DATE DEFAULT GETDATE(),

    CONSTRAINT FK_Prestiti_Utenti
        FOREIGN KEY (IdUtente) REFERENCES Utenti(IdUtente),
    CONSTRAINT FK_Prestiti_Libri
        FOREIGN KEY (IdLibro) REFERENCES Libri(IdLibro)
);
```

La tabella **Prestiti** collega **Utenti** e **Libri**, mantenendo l'integrità referenziale.

Normalizzazione

Definizione

La **normalizzazione** è il processo di **organizzazione dei dati in più tabelle** per:

- **eliminare la ridondanza** (evitare duplicati);

- migliorare la coerenza;
- semplificare la manutenzione del database.

In altre parole, la normalizzazione serve a **trasformare una tabella piatta** (una con molti dati ripetuti) in più tabelle **collegate tramite chiavi esterne**.

Esempio di normalizzazione

Supponiamo di avere una tabella "piatta":

TitoloLibro	NomeAutore	CognomeAutore
Dune	Frank	Herbert
Fondazione	Isaac	Asimov

Dopo la normalizzazione, diventa:

Tabella Autori

IdAutore	Nome	Cognome
1	Frank	Herbert
2	Isaac	Asimov

Tabella Libri

IdLibro	Titolo	IdAutore
1	Dune	1
2	Fondazione	2

Così si evita di riscrivere più volte lo stesso autore.

Gradi di normalizzazione (cenni)

1. **1NF (Prima forma normale)** → nessun campo multiplo o composto.
2. **2NF (Seconda forma normale)** → ogni colonna dipende interamente dalla chiave primaria.
3. **3NF (Terza forma normale)** → nessuna dipendenza transitiva (una colonna non dipende da un'altra colonna non chiave).

COLLATE

Definizione

Il termine **COLLATE** indica la **regola di confronto e ordinamento dei dati testuali** in SQL Server.

Serve per definire **come confrontare, ordinare e distinguere le stringhe**, tenendo conto (o meno) di:

- **Maiuscole/minuscole** (case sensitivity)
- **Accenti** (accent sensitivity)
- **Lingua/cultura** (collation specifica)

Esempio pratico

```
-- Esempio di confronto case-sensitive  
SELECT 'a' = 'A' COLLATE Latin1_General_CS_AS; -- FALSE
```

```
-- Esempio di confronto case-insensitive  
SELECT 'a' = 'A' COLLATE Latin1_General_CI_AS; -- TRUE
```

- CS = Case Sensitive
- CI = Case Insensitive
- AS = Accent Sensitive
- AI = Accent Insensitive

Applicazioni pratiche

- Uniformare il **collate** tra colonne di tabelle diverse per evitare errori in join e confronti (Cannot resolve collation conflict).
- Forzare un determinato ordinamento in una query:

```
SELECT Nome FROM Autori  
ORDER BY Nome COLLATE Latin1_General_CI_AS;
```

- A livello di database, si può impostare un **collate globale**:

```
CREATE DATABASE Biblioteca COLLATE Latin1_General_CI_AS;
```


Relazioni Gerarchiche in T-SQL

Definizione generale

Una **relazione gerarchica** descrive una **struttura ad albero**, in cui un record può essere “**genitore**” di altri record (“**figli**”).

Questo modello si usa per rappresentare **catene di comando**, **organigrammi**, **categorie e sottocategorie**, o qualsiasi struttura **ricorsiva**.

In T-SQL, tali relazioni si rappresentano tipicamente con una **tabella autoreferenziale**, oppure con una **tabella di collegamento (unione)** che mette in relazione gli elementi della stessa entità.

////DA INSERIRE RELAZIONE GERARCHICA **Relazioni gerarchiche**

Dover per esempio si ha una tabella con persone , dove l id nella tabella figlia si definisca una gerarchia della tabella persone, usiamo una tabella gerarchia dove abbiamo una tabella unione con id capo e id dipendente , se uno è il capo di se stesso semplicemente non vengo messo nella tabella

Casi di relazione (cardinalità)

1 → 0 o 1

Una persona **può avere o non avere** una mail.
È una relazione **uno-a-zero-o-uno**.

Esempio:

```
CREATE TABLE Persone (  
    IdPersona INT PRIMARY KEY IDENTITY(1,1),
```

```

    Nome NVARCHAR(100)
);

CREATE TABLE Mail (
    IdMail INT PRIMARY KEY IDENTITY(1,1),
    Indirizzo NVARCHAR(200) NOT NULL,
    IdPersona INT UNIQUE, -- Ogni persona può avere al massimo una mail

    CONSTRAINT FK_Mail_Persone
        FOREIGN KEY (IdPersona) REFERENCES Persone(IdPersona)
);

```

Se una persona **non ha una mail**, semplicemente **non viene inserita** nella tabella Mail.

1 → 0 o molti

Una persona può avere **nessun o più elementi associati** (es. più numeri di telefono, o più libri scritti da un autore).

Esempio classico: **Autori → Libri**

```

CREATE TABLE Autori (
    IdAutore INT PRIMARY KEY IDENTITY(1,1),
    Nome NVARCHAR(100)
);

CREATE TABLE Libri (
    IdLibro INT PRIMARY KEY IDENTITY(1,1),
    Titolo NVARCHAR(200),
    IdAutore INT NOT NULL,

    CONSTRAINT FK_Libri_Autori
        FOREIGN KEY (IdAutore) REFERENCES Autori(IdAutore)
);

```

La relazione è **uno a molti**: un autore può avere **molti libri**,
ma **ogni libro deve avere un autore** (quindi IdAutore **non può essere NULL**).

Nota importante:

Se volessimo rendere IdAutore **facoltativo**, dovremmo accettare anche libri “anonimi”, impostando IdAutore come NULL.

In tal caso la relazione diventerebbe **(0 o 1) → molti**.

Notazione grafica delle relazioni (Zampe di corvo / Crow's Foot)

Nella modellazione concettuale (diagrammi ER), le relazioni si rappresentano con la **notazione “zampa di corvo”**, che mostra il tipo di cardinalità tra le tabelle.

Simbolo	Significato	Esempio
1 — 1	uno-a-uno	Persona–CartaIdentità
1 — <	uno-a-molti	Autore–Libri
1 — O <	uno-a-zero-o-molti	Cliente–Ordini (un cliente può anche non averne)

Esempio visivo:

Specie 1 —< Animali

“Più animali possono appartenere a una sola specie”.

Esempio completo: Specie e Animali

Creazione tabelle:

```
CREATE TABLE Specie (  
    IdSpecie INT PRIMARY KEY IDENTITY(1,1),  
    NomeSpecie NVARCHAR(100)  
);
```

```
CREATE TABLE Animali (  
    IdAnimale INT PRIMARY KEY IDENTITY(1,1),  
    NomeAnimale NVARCHAR(100),  
    IdSpecie INT NOT NULL,  
  
    CONSTRAINT FK_Animali_Specie  
        FOREIGN KEY (IdSpecie) REFERENCES Specie(IdSpecie)  
);
```

Inserimento dati:

```
INSERT INTO Specie (NomeSpecie)  
VALUES ('Cane'), ('Gatto'), ('Cavallo');
```

```
INSERT INTO Animali (NomeAnimale, IdSpecie)
```

VALUES ('Fido', 1), ('Luna', 2), ('Spirit', 3), ('Bella', 1);

Ogni animale **appartiene a una sola specie**,
ma una specie può avere **più animali**.

T-SQL — Subquery

Cos'è una Subquery

Una **subquery** (o **query nidificata**) è una query **inserita all'interno di un'altra query**.

Serve per **recuperare dati** che vengono poi utilizzati come **condizioni di filtro**, **colonne derivate** o **sorgenti di dati** per un'altra query.

Una subquery può restituire:

- **Una singola cella** → 1 riga × 1 colonna (scalare)
- **Una singola colonna con più righe**
- **Una tabella intera** (più righe e più colonne)

Subquery scalare

È la forma più semplice: **ritorna un solo valore** (una cella).

Può essere usata ovunque sia ammesso un valore singolo, ad esempio in un WHERE, SELECT, o SET.

Esempio:

```
SELECT Titolo, AnnoDiUscita
FROM Film
WHERE AnnoDiUscita = (
    SELECT MIN(AnnoDiUscita)
    FROM Film
);
```

Restituisce il film più vecchio, confrontando con il **valore singolo** restituito dalla subquery. **Concetto chiave:** una **subquery scalare** si comporta come una **costante calcolata al volo**.

Subquery che restituisce una colonna

Se la subquery **ritorna una colonna con più righe**, si usa **IN**, **NOT IN**, o **ANY/SOME/ALL**.

Esempio:

```
SELECT Titolo
FROM Film
WHERE Titolo IN (
    SELECT Titolo
    FROM Libri
);
```

Restituisce i **film che esistono anche come libri**.

IN confronta il valore di una colonna con **una lista di valori** restituita dalla subquery.
È l'estensione del confronto scalare a un **insieme di valori**.

Subquery che restituisce una tabella

Quando la subquery restituisce più **righe e colonne**, può essere utilizzata:

- in una **clausola FROM** (come una **tabella virtuale**);
- o in una **CTE** (Common Table Expression).

Esempio:

```
SELECT F.Titolo, F.AnnoDiUscita
FROM (
    SELECT Titolo, AnnoDiUscita
    FROM Film
    WHERE AnnoDiUscita > 2000
) AS F
WHERE F.AnnoDiUscita < 2010;
```

La subquery crea una **tabella temporanea** filtrata (film post 2000) su cui poi si applica un secondo filtro.

Subquery vs. CTE

Entrambe permettono di scrivere query “intermedie”, ma ci sono differenze:

Caratteristica	Subquery	CTE (Common Table Expression)
Definizione	All'interno di un'altra query	Definita prima con WITH
Riutilizzabilità	Non riutilizzabile	Riutilizzabile nella stessa query
Leggibilità	Meno leggibile in query complesse	Molto più leggibile
Performance	Simile nella maggior parte dei casi	Talvolta più ottimizzata
Scopo	Limitata alla clausola in cui si trova	Valida per l'intera query

Esempio di CTE:

```
WITH Film2000 AS (  
    SELECT Titolo, AnnoDiUscita  
    FROM Film  
    WHERE AnnoDiUscita > 2000  
)  
SELECT Titolo  
FROM Film2000  
WHERE AnnoDiUscita < 2010;
```

È come una **vista temporanea**, valida solo durante l'esecuzione della query.

Esercizi ed esempi pratici

Esercizio 1 — Conteggio per Nazione

“Seleziona ciascuna nazione e il conteggio di persone nate in quella nazione”

```
SELECT  
    COUNT(*) AS [Count],  
    N.NomeNazione  
FROM ang.Persone P  
INNER JOIN geo.Citta C ON C.CittaId = P.CittaId  
INNER JOIN geo.Nazioni N ON N.NazioneId = C.NazioneId  
GROUP BY N.NomeNazione;
```

Usa **JOIN** per collegare tabelle e **GROUP BY** per aggregare i risultati.

Esercizio 2 — Persone con secondo nome e animali domestici

“Seleziona nome, nazione di nascita e anno di nascita di tutte le persone che hanno un secondo nome e almeno un animale domestico. Ordina per data di nascita decrescente.”

```
SELECT DISTINCT
  P.Nome,
  P.Cognome,
  N.NomeNazione,
  YEAR(P.DataDiNascita) AS AnnoDiNascita
FROM ang.Persone P
INNER JOIN ang.SecondiNomi S ON P.PersonaId = S.PersonaId
INNER JOIN anm.PersoneAnimali PA ON PA.PersonaId = P.PersonaId
INNER JOIN geo.Citta C ON P.CittaId = C.CittaId
INNER JOIN geo.Nazioni N ON C.NazioneId = N.NazioneId
ORDER BY AnnoDiNascita DESC;
```

Qui si usano **più JOIN** per collegare le entità, e funzioni come **YEAR()** per estrarre l'anno dalla data.

Esprimere i rapporti gerarchici tra una persona ed un'altra

Serve una tabella che esprima il rapporto di sudditanza di una persona rispetto ad un'altra.

Faccio la tabella persone e poi la tabella relazioni che contiene id della relazione, id capo, guarda foto telefono

Esempio con full union

Mettiamo caso di avere una tabella di clienti e una tabella di dipendenti (per esempio azienda), voglio avere un elenco di tutte le persone che siano clienti o siano dipendenti nella stessa query.

Dobbiamo mettere gli stessi campi nelle due select, devono essere dello stesso tipo però le colonne

Differenza tra is null e isnull

Nella condizione bisogna mettere is null staccato, mentre attaccato è il metodo che cambia e sostituisce i valori null

Subquery

Classica query affrontata

Select col1 , col2...

From tab1

Where col3= Costante da confrontare nel where

Quando imponiamo un = ci aspettiamo che dall'altra parte ci sia un valore che rimane costante, volendo posso fare una query che restituisce un valore ovvero una colonna una riga, per esempio con una top 1.

Esercizi con le subqueries

--> nomi delle persone nate nei 3 anni più recenti

--> prime tre persone in ordine alfabetico e poi esporle in ordine decrescente e caso in cui ci siano doppi

Offset e impaginazione

Per esempio se volessi avere i record dal terzo al sesto scrivo offset 2 rows fetch next 4 rows only

Esempio con due tabelle di numeri

Confrontarle e mettere 1 quando ce corrispondenza tra le due tabelle e 0 quando non ce corrispondenza tra i valori delle due tabelle

--Seleziona il numero totale di persone nella tabella persone

```
select count( p.Nome) as num from ang.Persone p
```

--Seleziona nome e cognome (in due colonne separate) di tutte le persone --con il nome che comincia per G e il cognome che finisce per O

```
select p.nome , p.cognome from ang.Persone p where p.nome like 'g%' and p.cognome like '%o'
```

--Seleziona nome, eventuale secondo nome e cognome --concatenati (chiama la colonna FirstLastName), --delle persone che si chiamano "Giovanni", "Alessandro" o "Adriano". --Se il Secondo nome non esiste, mostra una "x" (es. Viktor x Slicaru)

```
select CONCAT(p.nome , isnull(s.SecondoNome,'x') , p.cognome) as FirstLastName from  
ang.Persone p left join ang.SecondiNomi s on p.PersonalId = s.PersonalId where p.Nome in  
( 'Giovanni', 'Alessandro ', 'Adriano' );
```

```
SELECT p.Nome, p.Cognome, STRING_AGG(s.SecondoNome, ' ' ) FROM ang.Persone as p LEFT JOIN  
ang.SecondiNomi AS s ON p.PersonalId = s.PersonalId WHERE p.Nome = 'Giovanni' GROUP BY  
p.Cognome, p.Nome;
```



```
select p.DataDiNascita , n.NomeNazione from ang.Persone p join geo.Citta c on p.Cittald = c.Cittald
join geo.Nazioni n on n.NazioneId = c.NazioneId where n.NomeNazione like 'i%' group by
p.DataDiNascita , n.NomeNazione
```

--Seleziona l'anno di nascita (chiama la colonna birthyear) --più alto per ogni città per le persone --
che non hanno un Secondo nome select c.NomeCitta, max(p.DataDiNascita) as BirthYear from
ang.Persone p join geo.Citta c on c.Cittald = p.Cittald left join ang.SecondiNomi s on p.Personald =
s.Personald where s.Personald is null group by c.NomeCitta

```
select (p.Nome) from ang.Persone p where p.Personald not in( select p.Personald from ang.Persone
p join anm.PersoneAnimali pa on p.Personald = pa.Personald join anm.Animali a on a.AnimaleId =
pa.AnimaleId join anm.TipoAnimale ta on a.TipoAnimaleId = ta.TipoAnimaleId where ta.TipoAnimale
='gatto' ) order by p.nome
```

--Seleziona i valori distinti del nome di tutte le persone --che hanno più di un animale domestico e il
relativo conteggio select p.Nome, p.Cognome from ang.Persone p join anm.PersoneAnimali pa on
p.Personald = pa.Personald group by p.Nome , p.Cognome having count(pa.Personald)>1

ottavi di finale

*Seleziona nome e cognome (in due colonne separate) di tutte le persone che hanno il cognome che
comincia con B

*Seleziona la data di nascita della persona più giovane

*Seleziona il numero totale di persone nella tabella persone

Seleziona nome e cognome (in due colonne separate) di tutte le persone nate nell'anno 2003,
ordinate in ordine alfabetico decrescente per cognome

Seleziona tutti i nomi di battesimo distinti e ordinali in ordine alfabetico crescente

Seleziona tutte le persone che hanno il nome che non finisce con E

Seleziona nome e cognome di tutte le persone nate dopo il 2000 nella stessa colonna, chiamata
"giovincelli" (non usare "+" per unire le stringhe)

Seleziona la media degli anni di nascita di tutte le persone nella tabella

quarti di finale

Seleziona, per ogni anno di nascita, il numero di persone nate in quell'anno

*Seleziona nome e cognome (in due colonne separate) di tutte le persone con il nome che comincia per G e il cognome che finisce per O

Seleziona i valori distinti del nome di battesimo di tutte le persone che hanno il cognome che comincia per F

Seleziona nome e data di nascita della persona più giovane, se ha secondo nome null, mostra "-"

Semifinali

Seleziona i nomi e i cognomi, separati da spazio, nella stessa colonna (chiamata "class") per tutte le persone che hanno il nome con che ricorre più di una volta

Seleziona nome e cognome della terza persona più giovane della tabella table

Finale

Seleziona i nomi propri (chiama la colonna "nomi propri"), la media dell'anno di nascita e il massimo dell'altezza arrotondata al primo decimale di tutte le persone che hanno il cognome che non contiene "OS" o "CE", nate dopo il 1990 - la tabella deve avere come alias la parola "table"

Seleziona tutti i nomi propri che hanno meno di tre occorrenze, e il numero di occorrenze (chiama questa colonna "conteggio nomi"), escludendo le persone con secondo nome non valorizzato; aggiungi un'ulteriore colonna con il cognome che, a parità di nome, viene prima in ordine alfabetico

Seleziona i nomi propri (chiama la colonna "nomi propri") e la media dell'anno di nascita di tutte le persone che hanno il cognome che contiene "AR" o "DI", nate dopo il 1990 - la tabella deve avere come alias la parola "table"

ottavi di finale

* Seleziona ciascuna nazione e il conteggio di persone nate in quella nazione (chiama la colonna con il conteggio "Count")

Seleziona nome, cognome e numero di animali per ciascuna persona che ne ha almeno uno (chiama la colonna con il conteggio "PetSum"). Ordina per la colonna "PetSum" in ordine decrescente - in caso di parità, per cognome (sempre decrescente).

Seleziona nome, nazione di nascita e altezza arrotondata al primo decimale di tutte le persone che hanno un secondo nome e almeno un animale domestico - ordina per data di nascita decrescente

Seleziona nome, eventuale secondo nome e cognome concatenati (chiama la colonna FirstLastName), delle persone che si chiamano "Giovanni", "Alessandro" o "Adriano". Se il Secondo nome non esiste, mostra una "x" (es. Giovanni x Bertoglio)

Seleziona la città di nascita, l'anno di nascita e il cognome delle prime tre persone più giovani. Escludi le persone con data di nascita mancante.

*Seleziona i valori distinti di nazioni e anni di nascita (chiama la colonna birthyear) per le persone nate in nazioni che cominciano con "I"

seleziona il conteggio dei nomi distinti della tabella Person e il massimo del numero di animali posseduti da una persona

(la select deve avere una sola riga come risultato)

Seleziona l'anno di nascita (chiama la colonna birthyear) più alto per ogni città per le persone che non hanno un Secondo nome

Seleziona il numero e il tipo degli animali domestici, il nome e l'altezza approssimata al primo decimale (chiama la colonna "altezza" - non usare cast o convert) delle persone nate in Italia dopo il 1992, e che hanno il cognome che contiene la lettera "E"

Seleziona i valori distinti del nome di tutte le persone che non hanno gatti, ordina per nome decrescente

seleziona la media del numero dei gatti per nazione, per tutte le persone nate in Italia o che hanno il nome che comincia per "A" o "H" o "V"

Quarti di finale

Seleziona i valori distinti del nome di tutte le persone che hanno più di un animale domestico e il relativo conteggio

Seleziona la nazione e il numero medio di animali che hanno le persone nate in quella nazione - chiama la colonna con la media "PetCount" e convertila nel tipo nvarchar(255).

Seleziona nome+secondonome (nella stessa colonna "FirstMiddleName") e data di nascita della persona nata in Italia che ha più animali

Seleziona Città e Cognome di tutte le persone nate in Italia che hanno il cognome che inizia per "C" o "G" e che hanno almeno un animale. Ordina per cognome (decrescente)

Seleziona nome, cognome, paese di nascita e altezza approssimata alla prima cifra decimale delle persone senza secondo nome e senza animali

Semifinale

Seleziona il numero totale di gatti di persone non nate a Torino, che hanno il nome che comincia per "C", "B" o "G"

Seleziona i valori distinti dei nomi propri e della nazione di nascita di tutte le persone che non hanno secondo nome

Finale

scrivi due query diverse per selezionare nome, cognome e nazione di nascita delle persone che non hanno animali - escludi le persone senza data di nascita. Ordina per nazione (decrescente) e cognome (crescente)

